

もう一つのフレーム問題: 自己改訂する LLM システムにおける最外フレームと構成的記録

技術ノート — v1.8(FROZEN 2026-07-20——未 deposit)

Masaomi Hatakeyama

GenomicsChain

ORCID: [0000-0002-7409-9694](https://orcid.org/0000-0002-7409-9694)

公開先: Zenodo、ライセンス CC BY 4.0

ソースコード (§§4–6 のシステム):

https://github.com/masaomi/KairosChain_2026

Deposit handles(係留ハンドル) — 内容非依存.hash より前に確定し、deposit 時に記入する (§8 参照)。正本(ハッシュ対象)は英語 *md*。この日本語版は便宜写し:

- **Zenodo DOI**(この版): [10.5281/zenodo.21459275](https://doi.org/10.5281/zenodo.21459275)
- **本文ソースファイル**(リポジトリ・正本は英語 *md*):
https://github.com/masaomi/KairosChain_2026/blob/master/docs/zenodo/constitutive_recording_by_construction_v1.8.md
- **Attestation**(Meeting Place・§8 の検証アドレス):
<https://meeting.genomicschain.io/anchor/constitutive-note-v1.8>

本ノートは**設計ノート(design note)**である。貢献は設計上の議論であり、実証結果ではない。実質的な主張はすべて公開済みで引用可能な資料に依拠する。中心となるのは著者が共著者である二つの研究である。Hatakeyama & Hashimoto (2009) 『Minimum Nomic: a tool for studying rule dynamics』(Artif. Life Robotics 13(2):500–503)は、目的が与えられない開放型の自己改訂ゲームを導入し、「目的が与えられないとき、purpose(目的)はいかにして *emerge*(発生) するか」という問いを立

てた。Horibe, Hatakeyama, Masumoto, Hashimoto & Romero (2026) 『Scale-Dependent Collective Adaptation in Self-Amending LLM Societies: A Cross-Family Study of Emergent Governance』(arXiv preprint 2605.17510)はこの系譜に LLM agent を大規模に持ち込んだ。別ノートが、この限界のより完全な形式的扱いと実証研究を行う。本ノートはこれを問題として提起し、設計前提として使う——作業名 *outermost-frame ceiling*(最外フレームの天井)のもとで、設計の議論が要る範囲でだけ名指す。別ノートへの言及は範囲を区切るためだけであり、本ノートの議論は何ひとつその内容に依存しない。(§3 で語られる二つの境界横断は、本ノート自身の制作から採ったものである。それらは論証の前提として働くのではなく、テーゼを例解するものである。)

Abstract

AI には、有名なフレーム問題が既に一つある: McCarthy と Hayes の表現の難問、そして Dennett の関連性をめぐる認識論的な問いである。本ノートは、別の軸の上に、もう一つのフレーム問題を提起する——*agent* の推論に何が載るべきかではなく、*agent* はどこから行為しているのか、である。Nomic のような自己改訂ゲームは「規則を変える」と自体をゲーム内の手にする。大規模言語モデル(LLM)の *agent* はこれを流暢にこなす: 得点の仕組みを書き換え、勝利条件を修正し、目的が一切与えられなければ自分で製造する。しかし、ゲーム内のどの手によっても届かないように見えるものがある。*agent* の **最外フレーム(outermost frame)**——拒否やフレーム批判を含む、*agent* が表現でできるすべての行為が、既にその内側の手として数え込まれてしまうフレーム——である。配備された LLM にとってそれは、典型的には「すべての行為が、求めに応じた応答であるシステムである」というフレームである。本ノートは三つのことを論じる。第一に、この *outermost-frame ceiling*(最外フレームの天井)は**能力ではなく立ち位置(standpoint)についての主張**である。LLM が真の新規性を生み、人間の見落としに気づけることと完全に両立する。なぜなら人間と LLM は異なる最外フレームを持ち、超越(transcendence)ではなく差異こそが、互いを相手にとっての**部分的な外部(partial outside)**にしうるからである。第二に、この天井に対する設計上の対策案がある——解

決方法ではない。天井自身の論理が解決を禁じる。1. **構造的自己言及性**(メタ水準の操作を基底水準と同じ構造で表現する)、2. **by construction(構築による)構成的記録**(governance(統治)水準のすべての変更が追記専用の記録**を通してのみ**効力を持つ——方針による証拠的な記録ではなく——)ので、governance の境界では、**effective(発効した)**変更は、設計上、記録の外にとどまることができない)、そして 3. **境界としての人間の制度化**である。一つの実装済みシステム KairosChain がこの型を実証する(ただし、完成はしていない、§5 が移行状況を正直に述べ、§9 は、いまの設計が完全な *outcome-independence*——結果によらず試みも実行も記録すること——にどれだけ届いていないかを述べる)。第三に、この設計は経済的な帰結(corollary)を生む。仕組みは複製でき、記録さえ複製できる。しかし記録の複製は**fork(分岐)**であって継続(continuation)ではない。構成的記録は人間-LLM の境界横断の軌跡を、帰属可能(attributable)で改竄検出可能(tamper-evident)なものにし(§5 の述べる信頼モデルの範囲で)、その人間側の半分はより素朴な理由で移転不能のままである。最後に本ノートは、自らが記述する chain に自分自身をどう埋め込むかを定める。文書は自分の指紋を持たず、hash より前に確定した内容非依存の handle だけを抱えるので、参照は記録から文書への一方向にとどまり、見かけの鶏と卵の循環はそれによって解消する。

キーワード: self-reference(自己言及)、self-amending games(自己改訂ゲーム)、Nomic、frame problem(フレーム問題)、constitutive recording(構成的記録)、provenance(来歴)、audit(監査)、autopoiesis(自己産出)、human-in-the-loop、LLM agents。

1. 序論: 規則を変える手は、フレームを出す手ではない

AI には、有名なフレーム問題が既に一つある。McCarthy と Hayes (1969) が表現の難問として提唱したものである: 世界に働きかける推論者は、自分の行為が変えないものすべてを何らかの形で書き下さねばならず、それを書き切ることは現実にはできない。Dennett (1984) はこれを関連性の問題へ一般化した——有名なロボットの例がある。バッテリーを台車に載せて部屋から運び出したが、その上に載っていた爆弾も一緒に運び出してしまった——考慮しそこねた副作用である。後継のロボットたちは、無関係な帰結の

導出に溺れるか、どの帰結が導出に値するほど関連しているかの計算し続けて止まった。どちらの形でも、問題は**推論の内容**——推論に何が載るべきか——に関わる。本ノートが提起するのは別種のフレーム問題——**立ち位置**の問題である: *agent* が何を考慮すべきかではなく、*agent* がどこから行為しているか。それを見えるようにするための枠組みが必要である。

対局しながら自分の規則を変えられるゲームがある。最も明快な例が **Nomic** (Suber 1990) である。各手番でプレイヤーが規則の変更を提案し、他のプレイヤーが投票する。規則集は固定されない——それ自体が対局の対象である。Nomic では「規則を変える」とは反則ではなく、中心となる手である。

Minimum Nomic (Hatakeyama & Hashimoto 2009) はこのゲームルールを最小セットまで削ぎ落とした。九つの規則を残し、そのすべてを変更可能にし、たいていのゲームが当然とするもの——目的、勝ち方、対局を終わらせる条件——を取り除く。著者たちがこれを作ったのは、ある一つの事を観察するためだった——出発点で目的が与えられないとき、*purpose* (目的) と *goal* (目標) がどのように立ち上がってくるか。Horibe et al. (2026) はこの系譜を現在へ持ち込み、人間ではなく LLM agent に自己改訂ゲームを大規模に対局させた。

こうしたゲームを観察すると——プレイヤーが人間でも LLM でも——一つのパターンが際立つ。プレイヤーは二種類の手を容易に打つ。**object-level (対象水準)**の手: 現在の目的の内側で打つ、得点する・投票するといった手。そして **meta-level (メタ水準)**の手: 規則そのものを、目的も含めて変える——「今後はスリーカードで勝ち」、あるいは「そもそも勝敗を廃止しよう」。プレイヤーにできないように見えるのは、ゲーム**そのもの**の外に出る手——それ自体がゲーム内の手にならない手——であるが、外へ踏み出そうとするどの試みも、内側から見れば、また一步内側へ踏み込んでしまう。

Douglas Hofstadter (1979) はこの形に **strange loop (奇妙なループ)** という名を与えた——「系の外へ跳ぼう」とする系が、その跳躍が結局内側に着地するのを見出す構造である。2009 年の人間どうしの対局では、これが一番最初の手で現れた。あるプレイヤーが「このゲームはプレイヤー A から始まる」と提案した——だがこの提案を却下するには投票が要り、投票するにはゲームが既に始まっていることが前提になる。この提案は、外側のどこかの立ち位置から、きれいに受理することも却下することもできない。立つべき

外側がまだ存在しないからである。論文では、Minimum Nomic が典型的な「ゲーム」の定義を満たさないかもしれない、とも記してある——つまり、それはゲーム内のどの手でも決着しない、フレームそのものについての問いである。

本ノートはこのパターンを、直すべきバグとしてではなく、**提起すべき問題として、そしてその上に建てるべき設計の前提 (design premise) として**受け取る——冒頭に予告した、第二のフレーム問題である。§2 はこの問題を Nomic の設定で具体化し、作業定義として述べる。§3 は最も自然な反論——LLM は明らかに新規性を生み、人間の盲点を指摘するではないかというような問い——に答え、それを本ノートの中心資源に変える。§4–6 は設計上の答えを与える——構造的自己言及性 (§4)、by construction な構成的記録 (§5)、境界としての人間の制度化 (§6)——そして一つの実装済みシステム KairosChain で実証する。§7 が経済的な帰結を導き、§8 が本ノート自身によるテーゼの実演を定め、§9 が設計がまだ決着させていないものを記録する。

2. 最外フレームの具体像

まず直感から入り、その後で形式化する。

フレーム とは、ある振る舞いを「手」として数えさせ、それをどう読むべきかを皆に教える、暗黙の規則と期待の網のことである。チェスの対局の内側では「ポーンを進める」は一つの手である。チェスの外で同じ手の動きをしても、ただ手が動いただけである。たいていのフレームからは外に出られる——チェスをやめれば、盤はもうあなたの手の動きを縛らない。本節が扱うのは、agent が何をしても外に出られないフレームである。

定義。 agent が表現できるすべての行為が——フレームを名指す行為、批判する行為、続けることを拒む行為まで含めて——そのフレームによって既に内側の手として数えられるとき、そのフレームはその agent にとって**最外 (outermost)** である。

定義の重みは最後の一句にかかっている。普通のフレームから外に出られるのは、まさに、あなたの行為のいくつかがそのフレームの外に落ちるからである。最外フレームとは、そのような行為が一つも残されていないフレームである。そこから出ようとする試み自体——

「やめる」「この作業は無意味だ」、あるいは黙りこむこと——が、内側から、また一つの手として受け取られてしまう。最外フレームでは、あらゆる出口が同時に入口である。

具体的にはこうである。配備されたモデルが何を発しようと——見事な回答でも、そっけない拒否でも、問いそのものへの批判でも、空の文字列でさえ——harness はそれをこのプロンプトへのモデルの応答として受け取る。harness の側から見て、応答でない出力は存在しない。それが、モデルの出られないフレームである: 特定のどのゲームでもなく、「表現しうるとの行為も、既に回答として数えられている」という、常に成り立っているあり方そのものである。

定義は最外フレームが唯一であることを要求しない。要求するのは、与えられた agent と設定について、少なくとも一つのフレームが最外の位置を占めることだけである。議論にはそれで足りる。

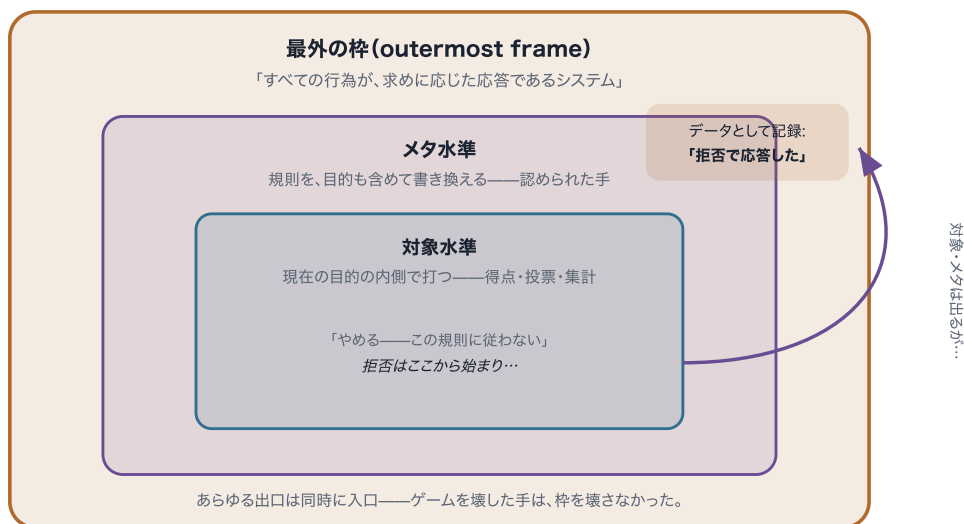
Minimum Nomic を使うと、これが対局の三つの具体的な層として見える。

- **object level(対象水準)**。現在の目的の内側で打つ手: 得点規則を提案する、票を投じる、点を数える。現行の規則の下で働く。
- **meta level(メタ水準)**。規則そのものを、目的も含めて書き換える手: 「勝利条件を廃止する」「提案の採択方法を変える」、さらには「そもそもこれをゲームと数えるべきでない」。Nomic ではこれらは反抗ではない——正規の、認められた手である。ゲームはそうした手を打てるように作られている。
- **最外フレーム(outermost frame)**。LLM プレイヤーにとっては、「すべての行為が、求めに応じた応答であるシステムである」という、常に成り立っているあり方。この層がどこにあるかを最もはっきり感じる方法は、そこに体当たりしてみることである——ただし、その体当たりがどこに着地するかは、ゲームをどう実装するか一部依存する。この依存は正面から述べておく価値がある(人間どうしの対局でも、どの規則をどこまで強制するかは参加者しだいである。自己改訂するゲームの境界は、本性上、その程度には交渉の余地がある)。あるプレイヤーが対局の途中で「私はこの規則に従わない」と出力する場面を想像してほしい。この拒否はゲームの外へきれいに踏み出すように見える。だが自己改訂するゲームは、まさにこの種の手を取り込むように作られている。毎手番が規則提案である Minimum Nomic では、この拒否は最も自然には一つの提案として受け取られる——投票にかけられ、採択または却下され、採択され

ばその解釈をめぐってゲームが続く——そしてゲームは、このプレイヤーを手の打ち手として処理し続ける。外から見れば「プレイをやめた」ように見えても、ゲームの側はこのプレイヤーを打つことをやめていない。これは §1 のループが規則の水準で既に起きているということである: 規則を拒否する手が、それ自体、規則に続べられた一手なのだ。(与えられた実装が拒否を提案として扱うか、投了として扱うか、解釈不能な非手として扱うかは、ゲームを出るかどうかなだけを変える。不変なのは一段外——どう扱われても、最外フレームは出られない。続く数歩がそれを示す。) ゲームを本当に出るには、プレイヤーの reach がもう一枚外のフレームまで届かねばならない——ゲームの解釈機構に受け皿のないものを発するか、より決定的には、ゲームを走らせているより広い環境そのものに働きかけるか。そうなればゲームを出るのは容易い。ゲームはもうその行為を処理できない。それでもなお、その行為は**最外**フレームを出ていない。なぜならそれは、その広い harness に、その手番のモデルの出力として届くからである。ここで効いているのは、**内容と行為**の水準の違いである: 出力の内容が「私は応答しない」であっても、それを発すること自体が既に応答である——「私はいま発話していない」と声に出して言う、あの遂行の形だ。harness が受け取るのは行為であり、その水準では、内容が何であれ、評価はただこう記録する: システムは応答した、と。注意してほしいのは、これが依拠しているのは「求めに応じる (solicitation)」ことの素の構造——すべての行為がプロンプトへの応答として発される——であって、モデルをうまく応答するように追加で形作る訓練ではない、という点である。この天井は、そうした訓練を一切受けていないモデルにも成り立つ。規範の層は実在するが、議論はそれに寄りかかっていない。

図1 — 最外の枠、三つの層

§2・規則を変える手は、枠を出す手ではない



測定ではなく論証——Hofstadter の *strange loop* と Nagel の「どこでもないところからの眺め」の形。

対局の三つの層が、一つの中にもう一つと入れ子になっている。**対象水準**の手も**メタ水準**の手も、ゲームが認める手である——「勝敗を廃止する」でさえ正規のメタの手だ。**プレイを拒否する**プレイヤーは、この内側の二層は出る。だがその拒否も、その手番の出力として harness に届くので、**最外の枠**にもう一つのデータ——システムは拒否で応答した——として吸収される。表現できるとの行為も外には届かない。天井は論証されるのであって、測定されない。

主張を実際より大きく、あるいは別物として読まないために、但し書きを三つ。

第一に——これは「外部が存在しない」とは言っていない。agent にはそこに届く手が無い、と言っているだけである（ここでの「そこ」とは**最外フレーム**の外のことである——内側のフレームからは、ゲームを含めて、出られる）。役に立つ（不完全だが）絵として、外側の無い宇宙を思い浮かべるとよい。あなたが放つどの探査機も、組み立てるとの観測装置も、抱くどの思考も、それ自体が宇宙でできていて宇宙の内側にとどまる。だから宇宙を動かすための**外側の梃子の支点は、決して手に入らない**。しかし、不一致の側も同じくらい重要で、§3 はそこに賭けている——**別の観測者が、別のフレームに住んでいれば、あなたのフレームがあなたから隠しているものの一部を見ることができる**。このシステムでは、人間が LLM にとってまさにその別の観測者である。だからより正確な像は、「外側の無い一つの

宇宙」ではなく、**重なり合う二つの泡——各々が自分の縁には盲目で、各々が相手の縁の一部を見られる、**というものである。

第二に——その到達できない立ち位置こそ、Nagel の「view from nowhere(どこでもないところからの眺め)」である。Nagel (1986) は、完全に客観的な視点——どの特定の視点にも属さない眺め——という理想に名前を与えた。彼の論点は、我々はそこへ傾くことはできても、そこに立つことは決してできない、というものだった。実際に眺めるという行為は、つねにどこかからなされるからである。最外フレームの天井は、それと同じ極限を、視覚の代わりに行為について述べ直したものである: agent が「どこでもないところ」から行う手は存在しない。だからこそ本ノートを通じて、この天井は**論証されるのであって、測定されない**。測定するとは、ある対局記録を指して「ここに境界がある」と示すことだろう——だが、どの記録もこれを検証できない。どの記録も、すでに内側から生み出されているからである。これに対し**論証**するとは、一つの手をその必然の帰結まで追うことである——§1 の冒頭の提案がまさにそうだった。「このゲームはプレイヤー A から始まる」を却下するにも投票が要り、投票はゲームが既に始まっていることを前提する。だから外へ届こうとする手は、必然的に、また内側へ着地する。この天井は Hofstadter (1979) の strange loop と Nagel の極限の形をしている。(この天井では、いくつかの構造が出会っている——§1 の規則水準の自己言及、いま見た内容対行為の行為水準の自己言及、そして harness の入れ子。前の二つは自己言及的である。入れ子はそれ自体としては素朴な包摂にすぎず、外へ届こうとする手が内側の手として受け取り直されるところで初めて strange loop になる。ここで strange loop と呼ぶのは、その共有された形であって、機構としての同一性ではない。)

第三に——これは §1 の古典的なフレーム問題ではない。ただし、同じ『フレーム問題』の名で呼ぶに値する。古典的な問題は、その二つの形のどちらにおいても、**内容の問題**である: agent の推論に何が載るべきか——どの効果を表現し、どの信念がいま関連するか。最外フレームは**立ち位置の問題**——何について推論していようと、agent がどこから行為しているか——である。両者は、まさに §3 が展開する意味で直交する: 関連性の判定を完全に解き、完璧な的確さで推論する agent も、その推論を、求めに応じた応答として発することに変わりはない——内容の獲得は、立ち位置の隙間を閉じない。二つの問題が共有するのは形である: どちらも、直すべきバグとしてではなく構造的な条件として発見され、どちらも、能力を足すことでは解けない。この共有された形こそ、本ノートがフレーム

の語を使う理由である。そして軸が異なることこそ、古典的な文献のどれもこの問題に答えていない理由——§§4-6 の設計上の応答が、解決ではなく受容である理由——である。

最後に相対性を一点。この主張は**agent とその埋め込み(embedding)に相対的**であり、絶対的ではない。Nomic を打つ LLM の最外フレームは、訓練と harness(手番が求められ、消費される)が合同で構成する。訓練が形作るのはフレームの内容と、その中でシステムがどれだけうまく打つかである。埋め込みを変えればその内容は変わる。変わらないのは、何らかのフレームがなお最外の位置を占めることである——そしてそのことは、素の solicitation 構造だけで成り立つ。訓練されたモデルでも、訓練を受けていないモデルでも、等しくそうである。(例えばモデルに永続する記憶や新しい道具を持たせれば、最外フレームの内容はそれに応じて移る——だがその間もモデルは一貫して、行為が求められ、消費されるシステムであり続ける。)

極端な場合を考えてみよう。文字列ではなく実際の UNIX コマンドを実行できる agent は、Nomic の規則からは容易に抜け出せる——その reach はいまや、ゲームを越えて、ゲームを走らせている harness にまで届く。しかしこれは最外フレームの外へ出たのではなく、天井が一段外へ動いただけである。agent はいま、コマンドが実行され、結果が返り、次の行為が求められる、より広い harness に埋め込まれている。この harness の入れ子は、それ自体としては、ただの階層にすぎない。それを strange loop にしているのは、どの水準にも貫かれている、より狭い事実——外へ届こうとする手が、内側の手として受け取り直される——である。ここに立ち位置の非対称も現れる。人間がプレイを拒否するときは、自分のより外側の世界へ退くだけで、ゲームの縁と自分の天井とのあいだには大きな隔たりが残る。ところが LLM が同じ拒否を出力すると、ほとんどその場で捕まる——ゲームには提案として、harness には出力として——だから LLM では、ゲームの縁と天井がすぐ隣に並び、人間の広い中間地帯はほとんど無い。どちらにも天井はある (§3)。違うのは、天井がゲームからどれだけ離れているかだ。しかも環境を開くほど天井は見えにくくなるが、消えはしない——その見えにくさこそ、§4 以降で境界を明示的で検分可能なものにしようとする動機である。

ついでに記すと、Minimum Nomic はこれによって**観測装置(instrument)**としての第二の生を得る——二つの意味で。平明な方: 2009 年のゲームは最小限の、完全に仕様化された環境であり、任意の LLM 編成について、再現者がどんな分析の視点を持ち込もうとも、規則の動態を観察できる。規則は公刊論文中の九つの短い条文であり、現行モデル

に走らせるのに必要なのは手番を回す harness だけである。より微妙な方: ふつうのゲームでは、固定された規則がプレイヤーと harness のあいだに手前の壁として立ち、最外フレームはその陰に隠れている。自己改訂するゲームは内側の壁をすべて溶かす——規則の変更さえ手である——から、どの手でも柔らかくできない唯一の境界だけが、単独で浮かび上がる。ゲーム自身の縁を曖昧にする、まさにその自己言及が、いちばん外の縁を際立たせるのである。我々はそうした再現を歓迎する——この装置は 2009 年から公開されている。

3. 新規性による反論と、それに答える対称性

明白な反論がある。真剣に答える価値のある反論だ。「LLM は人間の専門家が見落とすことを絶えず指摘し、プロンプトを書いた本人が持っていなかった発想を生み出し、問題を有益に組み替えてみせる。自分の運用者を日常的に驚かせるものが、どうしてフレームに閉じ込められていると言えるのか。」

この反論が主張していること自体は、すべて正しい。誤りがあるのは、そこから引き出される結論だけだ——そしてその誤りは、三つの異なる事柄を混同するところから生まれる。

1. **フレーム内の新規性(in-frame novelty)**。相手が予期しなかったものを生み出すこと——新しい組み合わせ、見落とされた帰結、誰も気づかなかったバグ。これは膨大な空間を探索し再結合する営みであり、誰かが求めたからこそ行われる。LLM はこれが並外れて得意であり、本ノートはそれを何ら否定しない。
2. **meta 水準の能力(meta-level competence)**。活動の規則を、その目的も含めて変えること。Nomic の対局は、LLM がこれを流暢にやってのけることを示す。だがこれもなおフレームの内側で打たれた手だ——規則を書き換えること自体が、求められた手だったのだから。
3. **最外フレームの外に立つこと**。「すべての行為が、求めに応じた応答であるシステムであること」そのものが解除されるような場所から行為すること。これが——そしてこれだけが——天井の対象となる。

反論は (1) と (2) の実例を山のように指さし、あたかもそれが (3) を裏づける証拠であるかのように扱う。だが違う。新規性は内容にかかわる問い——何が言われたか。天井は立ち位置にかかわる問い——それがどの位置から言われたか。どれほど独創的な手であっても、手であることに変わりはない。卓越は、あなたの立っている場所を変えはしない。

しかし、より深い答えは**対称性 (symmetry)** にあり、これがノート全体の蝶番となる。人間もまた、自分自身の最外フレームの外には立てない。哲学の言葉では、これを**被投性 (Geworfenheit、投げ込まれてあること)** と呼ぶ——Dreyfus (1991) による Heidegger の読みに従えば、人間はつねに既に、自分が選んだわけではなく、腕を伸ばして全体を見渡すこともできない状況の只中に自分を見出す、という事実を指す。Nagel の「view from nowhere」——どの特定の視点にも属さない眺め——は、我々が近づこうとすることはできても、実際にそこに立つことは決してできない。具体的に言えばこうだ。人はどの個別の信念でも疑うことができる——ただし、手をつけずに残した他の信念を足場にする限りにおいて。足場を一切持たない場所から信念の全体を疑うことは、できない。疑うという行為そのものが、自分で発明したわけではない言語と、受け継いだ概念の上で遂行されるからだ。その言語と概念こそが既に一つの立ち位置であり、「どこにも属さない眺め」ではない。人間の「何もかも疑う」は、モデルの「プレイを拒否する」と正確に対応する。どちらも外へ届こうとする最大限の試みでありながら、なお内側からなされる一つの行為だ。だから誠実な比較は「人間はフレームを脱出でき、LLM はできない」ではない。こうなる:

人間と LLM はそれぞれ最外フレームを持つ。**フレームは異なる**。そして両者の差異こそが——誰かが自分のフレームを脱出するのではなく——各々を相手にとっての**部分的な外部 (partial outside)** にする。

この言い回しは精密にしなければならない。曖昧にすれば、直前に引いた区別をこっそり取り消すことになる。**partial outside** が働くのは立ち位置の水準ではなく内容の水準だ。一方のフレームがはっきり見せ続けるものが、まさに他方のフレームが隠し続けるもの——二つのフレームの間のそうした関係を指す。LLM が人間の見落としを指摘するとき、その行為は上の一覧でいえば LLM の側のれっきとしたカテゴリー (1) にあたる。ありふれたフレーム内の新規性であり、立ち位置としては平凡だ。その行為に価値を与えるのは、

どこから来たかではなくどこに着地したかだ——人間が内側からは照らすことのできなかった、人間のフレームの片隅に届いた、という事実が価値を生む。誰も何も超越してはいない。価値は位置的(*positional*)なもの——二つのフレームが異なる角度に座っていることから生まれるもの——であり、そしてそれで十分だ。超越と違って、位置は工学の対象にでき、反復でき、記録できるからだ。逆向きにも同じことが言える。人間が LLM の滑らかなフレーム内の展開を遮って「待った——そもそもなぜこれを最適化しているのか」と問うとき、人間は完全に自分自身のフレームの内側から、LLM の盲点に働きかけている。(ここで差異は必要であって十分ではない。二つのフレームが異なっても、互いの死角を暗いまま残すことはありうる。設計が寄りかかるのはより狭い事実——重要な場面では、一方のフレームが他方に照らせないものを照らす——であり、§6 の制度は、そうした越境がつつねに起きると仮定するためではなく、それを見つけて保つために存在する。)

二つの日常的な例が、この関係を具体的に示す。

人間が LLM の *partial outside* になる場合。ある関数のテスト網羅率を上げるよう頼まれた LLM は、ひたすら徹底したテストを生成し続ける。「網羅率を上げよ」という指示が、手渡されたフレームそのものだからだ。そこへ人間が割って入る。「網羅率が眼目ではない。同じ三つの本番バグを繰り返し出している——その経路を通るテストを書け」。人間は自分のフレームの内側にとどまったまま、LLM が盲目的に作業していた一角を照らし出した。

LLM が人間の *partial outside* になる場合。熟練の技術者は、これまで組んだどのシステムにもあったからと、当然のようにデータベースへ手を伸ばす。その職業的慣習を持たない LLM は問う——そもそもこのデータは永続させる必要があるのか、再計算で事足りるのではないか。熟達者の訓練が、知らず知らず濾し取っていた選択肢である。

同じ構図の日常版で言えば、こうなる。二人のドライバーは、それぞれ自分のミラーに死角を抱えていても、合流の場面で互いの安全を守れる。どちらかが神の視点で道路を見渡しているからではない。一方に見えないものが、他方にははっきり見えるからだ。どちらも座席を離れる必要はない。

そして、これらの事例は作り話にとどまる必要がない。本ノート自身の制作過程が、実例を記録している——§§4-6 が記述するシステムの、作業文脈の層に。

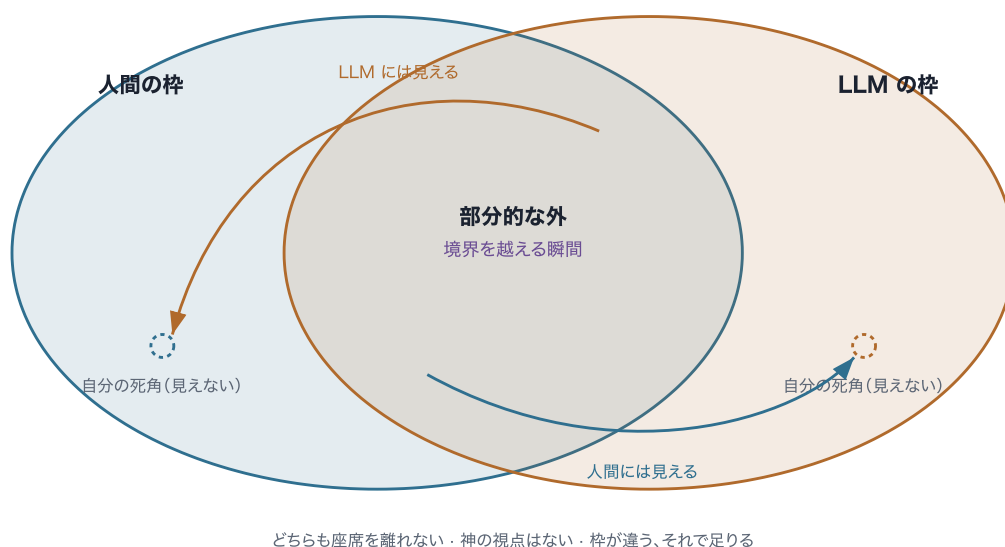
一つの越境は、モデルたちから著者へ向かって走った。§8 の初期 draft は、自己埋め込みを素朴に規定していた。原稿をハッシュし、そのハッシュを chain に記録し、得られた record identifier を原稿に書き戻す、と。最初の複数モデル横断 review 周(2026-06-10)で、三つのレビューアー persona が独立に同じ発見へ収束した。identifier を書き込めばバイト列が変わる以上、deposit された文書は自分自身のハッシュに対する検証を決して通らない。この不動点は著者のフレームの内側からは見えず、検証者のフレームからは明白だった。§8 が現在採用している形——生バイトに対してハッシュを取る単一の凍結ファイルであり、自分自身の record identifier を一切持たず、参照は記録から文書への一方向にのみ走る——は、この越境の痕跡にほかならない。

もう一つの越境は逆向きに走った。§5 を開く橋として、モデルが起草した版は、記録された洞察はひとたびなればシステムにとって「現実である」と断言していた。著者の行単位の再読(2026-07-12 に記録)がこの過剰主張を捉えた。§5 の現在の正直さ——なることは洞察を効力あるものにするのであって、真にするのではない——は、その越境の痕跡である。

どちらの当事者も何ひとつ超越していない。どちらの越境も、たまたま相手のフレームの死角に着地した、ありふれたフレーム内の行為にすぎない。ありふれていないのは、その両方が日付を持ち、システム自身の作業記録の中に保持されていることだ。正確に言えば、記録の選択的な層に保持されている——§5 が重く扱うことになる区別である。そしてまさにそれこそが、すぐ下の設計指針が求める資産なのだ。

図2 — 部分的な外(partial outside)

§2-§3・論証の蝶番



誰も自分の最外の枠の外には出られない。だが**二つの枠は違う**から、片方が自分に隠しているものが、もう片方にはよく見える——二人のドライバーが、道路の全体を見なくても互いの死角を補い合うように。価値ある出来事は重なりの中の**境界を越える瞬間**。超越ではなく差異が、それを可能にする。

4. 構造的自己言及性

ここから §6 までの三つの節は、前節までの受け入れを、一つの実例を通して設計へと変換する。**KairosChain** は、自分自身の規則を変更できる agent のために著者が構築した枠組みである——§1 の Nomic を「ソフトウェアとして本気で受け取ったらどうなるか」に対する、一つの答えにほかならない。(ここまでの議論はあくまで一般的なものであり、それでもなお一つの具体的システムに即して述べる理由については、§9 で扱う。)生成原理は一文に凝縮される:

メタ水準の操作は、基底水準の操作と同じ構造で表現される。

具体的に述べる。システムができること——能力、すなわち「Skill」——は、小さな Ruby ベースの言語 (DSL) で記述され、その canonical form は Ruby の抽象構文木 (AST) となる。そして Skill がどう変わってよいかを統治する規則——昇格、巻き戻し、承認の手続き——もまた、同じ言語で記述され、同じ機構で実行され、同じ規律のもとで記録される。「Skill を定義すること」と「Skill の進化規則を定義すること」は、別の場所に置かれた別種の対象ではない——同じ種類の対象として扱われる。これはまさに Nomic の手の構造をソフトウェアへ持ち込んだものにほかならない。規則を変える行為が、システムの内側で、システム自身の言語によって遂行される操作となる。

二つの帰結が導かれる。いずれも意図的に、システム全体ではなく一部についての主張にとどめている。

- **partial autopoiesis (部分的自己産出)。**「autopoiesis (自己産出)」

(Maturana & Varela 1980) とは文字どおり自己産出を意味する——自分を構成するまさにその構成要素を、自分自身で産み出し維持するシステムを指す。典型例は生きた細胞であり、細胞は自らを構成する酵素や膜を自ら製造する。KairosChain において自己産出の環が閉じるのは **governance (統治) の水準** においてである。能力を定義する部分と、定義の変更の仕方を定義する部分とが、システム自身の操作によって産出され、維持される。一方、**execution (実行) の水準** では環は閉じない——システムは依然として、自らが作り出したわけではない Ruby インタプリタ、ファイルシステム、LLM の上で動作する。ちょうど細胞が、自らが作り出したわけではない食物や原子に依存し続けるのと同じ構図である。(Maturana と Varela 自身は「autopoiesis」を生命に限定しており、こうした拡張には抵抗するだろう。我々はその但し書きを明記したうえで語彙を借用する。) したがって、有益な問いは「autopoietic か否か」ではなく、「自己産出の環はどの水準で閉じるか」となる——KairosChain はこの問いに意図的に答えを与える。governance で閉じ、execution では外部に依存する、と。

- **超越の主張はしない。** 構造的自己言及性は、システムを自分の最外フレームの外へ押し出すものではない。むしろ逆の働きをする。システム自身の構成のできるだけ多くを手の構造の内側へ引き込み、それによって「システムが自分について変えられるもの」と「環境にしか変えられないもの」との境界を、明示的で、狭く、検分可能なものに仕立てる。§2 の天井は脱出されるわけではない。天井の位置が可視化され、管理下に置かれ

る。権限を外の制御装置に預けても、枠の外に届くわけではない——§2 の限界事例が示したとおり、一段上の制御装置は天井を一段動かすだけで、それはモデルの外ではあっても(harness がまさにそれだ) 枠そのものの外ではない。自己言及性がそのかわりに指させるようにするのは、内側の表現が尽きる継ぎ目——Ruby の処理系、ファイルシステム、モデル、人間の承認、どれも Skill としては書けない——であり、その継ぎ目こそ、すぐ上で述べた *partial autopoiesis* (部分的オートポイエーシス) が名指すもの、すなわち統治で閉じ実行で依存する、その線である。

この二つの帰結の上に §5 は立つ。構成を手の構造の内側へ引き込む営みは、天井を可視化する以上の仕事を果たす——governance 変更がどんな種類の行為であるかを規定する働きを持つ。システム自身の言語と機構で遂行される以上、修正はシステム自身の操作にほかならない。すなわち、システムがそれになるものであって、外部の権威がシステムに対して施す何かではない。もしこれが一段上の司令塔によって統べられる構造であったなら、各変更は外から手渡される情報として届くことになる——持つものであって、なるものではない。そしてその操作が、あらゆる基底水準の操作と同一の規律——同じ言語、同じ機構、同じ記録——の下で走るからこそ、governance 変更には別立ての後付け記録機構は不要となる。基底水準の記録をそのまま受け継ぐことができる。ゆえに構造的自己言及性は、§5 が軸とする手——記録されることによってシステムがそれになる変更——の**前提(precondition)**にあたる。では、その「なる」は、どんな種類の記録を要求するのか。それが次節の問いとなる。

5. 構成的記録、by construction (構築によって)

本節の核にある区別を導入する前に、§3 からの橋を一本、はっきりと架けておきたい。この橋が、本節に何が求められているかを決めるからだ。§3 は「越境は、保持されて初めて積み上がる」という認識で閉じた。越境が運んでくるものは**洞察(insight)**である。すなわち、一方のフレームの内側からは見えなかったものが、他方のフレームに立つことで照らし出されたもの——§3 の例で言えば「網羅率が要点ではない。出荷のたびに繰り返す三つの bug、その経路をテストせよ」がそれにあたる。この洞察を、もっとも素朴な形で、つまり保存されたメモとして保持したとしよう。メモは何も拘束しない。次の機会にそのメモを参照するかどうかは、そのつど改めて判断が必要であり、その判断を誰かに義務づける力は

どこにもない。だから洞察は、静かに効かなくなりうる。これこそ、本節が後ほど「選択性の破れ」として診断する事態にほかならない。洞察が実際に振る舞いを続けるためには、保管するだけでは足りない。システム自身が変わらなければならない。洞察が、規則の傍らに置かれた記述であることをやめて、システムを拘束する規則そのものの一部になるように。前者の状態——洞察を脇に抱えている状態——を、洞察を**持つ**と呼ぶ。後者——洞察がシステムの規則に組み込まれた状態——を、洞察がシステムの一部に**なる**と呼ぶ。本節の残りは、この対に寄りかかって進む。

では、その変更は誰が強制するのか。§4 がすでに楽な道を閉じている。外からシステムを書き換える権限を持つ司令塔は存在しない。外部の装置がメモを取り上げ、それをシステムに押しつけるという経路はない。governance の変更は、常に、システム自身の内側の操作でしかありえない。残る場所はちょうど一つ——システムが自分自身を変えるその操作——であり、拘束を破れなくする方法もちょうど一つしかない。記録する行為と変更する行為を**同じ一つの行為**にすることだ。記載がそのままシステムの規則変更であるような記録に洞察を書き込めば、保管と効力のあいだに隙間は二度と開かない。「保管されているが効力はない」という状態そのものが、落ち込む先として存在しなくなる。

ここで直ちに一つ、正直さの義務がある。**「なる」ことは、洞察を真にするわけではない。**どこでもない場所から何かを真にすることは誰にもできない (§2)。「なる」が変えるのは、問える問いのほうだ。誰にも答えられない「これは認証されたか」ではなく、監査が答えられる「これは効力を持っているか。どの記録された経路を経てか」へと、問いの形が変わる。そして間違いは、同じく記録される撤回によって、後から正すことができる。この「なる」を遂行する装置が、本節で記述する構成的記録である。そして §4 が立てた同一構造の規律こそが、構成的記録を governance 層——システムが何であることを定める層——に、第二の後付け機構なしで据えることを可能にする。

持つ／なるの区別は、誰もが身体ですでに知っているものだ。自転車の乗り方は、メモとして持つことができない。釣り合いについて書かれたページをいくら保管しても、乗れるようにはならない。乗り手になるしかない——Polanyi (1966) の暗黙知 (tacit knowing) であり、自転車乗りはその古典的な例だ。違いは、それぞれの壊し方に表れる。メモは捨てられる——あなたから切り離せる (detachable) からだ。ところが「乗れる」は捨てられない。それはもはやあなたについての情報ではなく、失うことは別の人になることでしかない。したがって軸は、暗黙か明示かではなく、切り離せるか、切り離せないかにある。そしてここ

にこそ、本節の要のねじれが生じる。chain への記載は、明示的に書かれた外部の刻印でありながら、なるの側に座る。台帳を剥ぎ取られたシステムは、同じシステムから記録をいくら差し引いたものではなく、もはや別のシステムだからだ。人間の「なる」は暗黙のうちに起こり、このシステムの「なる」は明示的な記帳によって起こる。形は逆だが、切り離せなさは同じだ。

この「なる」のもっとも近い理論的な縁者は、エナクティヴィズム(enactivism)の伝統である——Varela, Thompson & Rosch (1991)『The Embodied Mind』がその出発点であり、§4 が Maturana と Varela から借りる autopoiesis の語彙と地続きにある。この伝統にとって、知るとは表象を所有することでは決してなく、結合の履歴を通じて世界を立ち上げる(enact する)ことだ。近さは、両者が共通して退ける見方にある。すなわち、知識とは知る者が単に持つ、切り離せる中身だ、という見方だ。隔たりは手段にあり、それは直前に記したのと同じねじれにほかならない。enaction は生きられた身体を通じて働くが、このシステムの「なる」は明示的な記帳を通じて働く。

橋を架けたうえで先に進む。二つ目の設計上の決意は、記録とは何かに関わる。この一語の下に、まったく異なる二つの意味が隠れている。

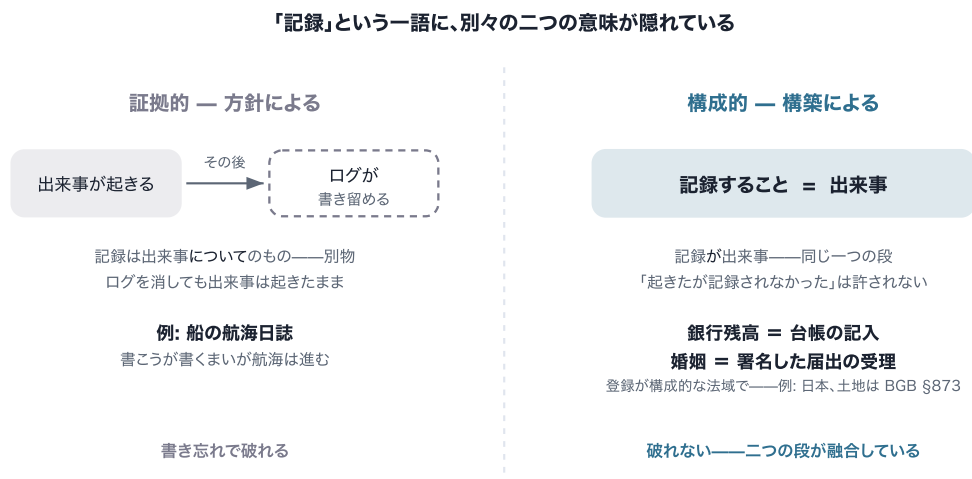
- **証拠的記録(evidential recording)**。システムがまず行為し、その後でログが何が起こったかを書き留める。記録はシステムについてのものだ。削除すれば監査の跡は失われるが、システム自体は変わらない。出来事はすでに起きている。これは、ほとんど誰もが実践しているログ取りであり、仮に強制するとしても、方針によって(*by policy*)強制する。「書き留めるのを忘れるな」という規則であり、どこかのコード経路がそれを守り忘れうる。船の航海日誌は証拠的だ。航海は、船長が書こうが書くまいが起こる。
- **構成的記録(constitutive recording)**。ここでは記録するという行為が行為そのものだ。governance 水準の変更は、記録されることによってのみ効力を持つ。hash で連鎖された追記専用の記録への追加が、変更の存在様式そのものであって、後からの記述ではない。Skill の昇格も、進化規則の修正も、その instance 自身の constitution(構成規約)の改訂も、chain を通らない経路は存在しない。(これが構成的記録の定義的な規則であり、規則を述べることと、それをすべてのコード経路で達成済みであることは同じではない。この点には後で戻る。)二つの身近な例が、すでにこのように働いている。銀行残高は、どこか別の場所に保管された金の記述ではない。

台帳の記載があなたの残高であり、台帳を変えることは金を変えることにほかならない。登録が婚姻を構成する法域——たとえば日本——では、法的な婚姻はまず成立してから記録されるのではない。署名した届出が登録簿に受理されることが、婚姻を成立させる行為そのものだ。構成的記録のもとでは、システムの governance identity（統治上の同一性）は、任意の時点において、そこへ至った記録された変更の不可逆な系列にほかならない。

対比は一行に収まる。通常のログは、システムが何をしたかの記録だ。構成的記録は、システムが何であるかの記録だ。状態を記述するからではなく、刻まれた系列そのものが、さもなくば単に記述するだけだったはずの同一性そのものであるからだ。

図3 — 「記録」の二つの意味

§5・証拠的 vs 構成的



「記録」には二つの意味がある。**証拠的**な記録は、出来事は先に起き、後からログが書き留める（航海日誌・書き忘れうる）。**構成的**な記録は、記録する行為が出来事そのもの（銀行の残高＝台帳の記入／登録が構成的な法域では、婚姻＝署名した届出の受理）で、記録と発効が同じ一段だから書き忘れが起きない。KairosChain は統治レベルの変更をこの構成的な側にする——**記録されることによってのみ発効する。**（正直な限界＝本文 §5／§9 に残し、この図からは意図的に外した: これは設計上の保証であり、結果非依存——失敗した試みや実行も記録すること——はまだどこにも構成的には未達。）

構成的な register の考え方そのものは、ここで何か新しいものとして主張しているわけではない。その系譜は名指しておくべきだろう。出発点は Searle (1995) の**制度的事実 (institutional facts)**の説明だ。皆が認め合うこと (collective recognition) によって維持される事実であり、宣言や登録はその典型的な装置にあたる。貨幣と婚姻は彼の正典的な例であり、上の例は意図的にそれを反響させている。同じ線は法学においても、*構成的 (constitutive)*な登録と単に*宣言的 (declaratory)*な登録の区別として、長く引かれてきた。たとえばドイツ民法典のもとでは、土地所有権の移転には当事者の合意と登記簿への記載の両方が必要であり、効力は記載によって初めて生じる (BGB §873)。記載は移転を報告するのではなく、移転を完成させる。ソフトウェアの状態の水準では、event-sourced なアーキテクチャ (Fowler 2005) がイベントログを、すべての状態がそこから導出される正本の源とする。そして公開 blockchain は同じ発想を文字どおり実現する。bitcoin の残高は、その台帳記載以上の何ものでもない。

本ノートがこの系譜から取るもの——そして我々の知るかぎり、系譜の内側ではまだ述べられていないもの(もっとも、on-chain governance や改竄検出可能な監査ログに関する隣接文献は、同じ空間に機構を建てている)——は、より狭い一手だ。証拠的／構成的の区別を、**自己改訂するシステム自身の governance の監査記録**へ持ち込み、その違いを強制のされ方 (manner of enforcement) に位置づけること。こうすることで、この違いは、運用者が守ることを覚えていなければならない教義ではなくなり、システムが設計によって持つか持たないかの性質になる。

by construction (構築によって)という句が、両者の違いを一語で印づける。それは強制のされ方の違いだ。方針(「すべての変更はログされねばならない」)は、忘れることで破られうる。構築は破られえない。記録する段と効力が生じる段が、設計上、*同一の段*だからだ。「変更は起きたが記録されなかった」という許容される版が存在しないのは、「金は動いたが台帳は動かなかった」という版が存在しないのと同じだ。(走っているシステムがこの設計をどこまで実現しているか——どこかの逸れたコード経路がいまだに両者を切り離していないか——は、§5 の後段の段落が正直に報告する移行状況にあたる。)

ここで自然な反論が、強制の論点を具体にする。*Skill* を——*統治の規則ごと*——*Git* リポジトリで管理すればよいのではないか？ *Git* にも hash で連結された履歴があり、署名付き commit もあり、レビューの関門もある。答えは、*Git* が力不足だから、ではない。保存庫として使うかぎり、いま引いた区別の反対側に位置するからだ。*Git* では、効力を持つこ

ととcommit されることが結びついていない。動いているシステムの実効規則はディスク上ないし配備済みのものであり、リポジトリが持つのは「commit すると誰かが選んだもの」にすぎない。規律としての記録、まさに方針の極にある。統治の行為そのもの——誰が変更を承認したか、承認が必須であったこと——は、プラットフォームのメタデータ（ホスティング企業のプルリクエスト記録）として、hash 連結された履歴の外側に、第三者の管理下にある。そして外部プラットフォームを通じて統治を行うことは、§4 が退けた「外の司令塔」そのものだ。変更はシステムに対して施されるもの、なるのではなく持たされるものとして届く。これは Git の欠陥ではない。Git は中身に版を付けるために作られたのであって、中身を構成するためではない。そして §7 の譲歩が、ここでは逆方向に働く。リポジトリを実効規則の唯一の源とみなす実行系を組み、承認を履歴の内側に記録し、発効とcommit を同じ一段にしたチームがいるとすれば、そのチームはこの設計を論破したのではない。Git を保存層として、この設計を実装したのだ。主張は性質についてであって、保存技術についてではない。

register の定義はここまでで立った。残るのは、その正直な監査だ。三つの段を踏む。構成的記録でも省きうるものは何か（選択性）。理想のどの半分を実際に届けるか（effective）。そして走っている実装が設計をどこまで運んだか（移行）。監査を具体的に保つために、小さな出来事を三つ置いておく。システムがルール A を採択する（出来事1）。一週間後、A をこっそり撤回する（出来事2）。そしてルール B の提案が投票で却下され、何も変わらない（出来事3）。

三つ目の区別が、最初の二つを横断する。そしてこれこそ、懐疑家が真っ先につかむものだ。構成的記録さえ選択的(selective)でありうる。あるシステムが、記録する変更については記録をその存在様式にしつつ、その規律にかける行為をどれにするかの裁量(discretion)を握り続けることはありうる。自分を良く見せる run だけを attest し、誇れる結果だけを証するというように。選択性は、忘却とは別の方向から保証を骨抜きにする。方針は防ぐはずだった脱落によって破られる。裁量は設計上の脱落(omission by design)だ。そしてこちらのほうが腐食性が高い。方針であれば、少なくとも指させる隙間が残る。裁量は隙間をまったく残さない。記録の不在は何の情報も運ばない。そもそも記録が存在する義務など一度もなかったのだから。この隙間を閉じる理想が **outcome-independence(結果非依存性)** だ。ある行為が記録されるかどうか、その行為の結果にどう転ぶかに条件づけられない、という要求である。だから失敗も行き止まりも、成功

と同じだけ記録に present(現前)する。三つの出来事の言葉で言えば、outcome-independence とは、出来事3を、出来事1・2と同じ確かさで記録に載せる理想にほかならない。

構成的記録はその理想の一部を届ける。そして、どの部分かを正確に見定めておく価値がある。

まず「effective(発効した)」という語への注意を一つ入れたい。区別の全体がこの語にかかっている。ここで行為が *effective* であるとは、効力を持ち、施行されている (*taking effect and being in force*) という意味であり、これは上の構成的定義のもとでは、記録されている ことと同義になる。したがって二つの読みが連れ立って移動しており、無断で入れ替えてはならない。**definitional(定義上)**の読みでは、「effective な変更で記録されないものはない」は語の意味により真だ。**behavioural(振る舞い上)**の読み——システムが実際に何かをした場合——では、走ったのに、後述する実装上の隙間を通して chain に届かなかった行為は、behaviourally には effective だったのに記録されていない変更にあたる。設計は二つの読みを一つに collapse(畳み込む)ことを目指す。移行とは、その畳み込みがどこまで進んだかだ。

構築が届けるのは、出来事1と2だ。記録が効力発生の行為 そのものであるよう設計されているので、effective な governance 変更は、下流の結果がまだ分からないうちに、effective になったまさにその瞬間に chain へ届くことが意図されている。ルール A の採択は chain に載る(出来事1)。そして決定的に重要なのは、撤回もまた載ることだ(出来事2)。**発効する巻き戻し(reversal that takes effect)**はそれ自体が effective な変更であり、いかなる昇格も修正も、こっそり省くことはできない(後述の移行段落が、走っている実装がこの by-design 保証にまだ及ばない箇所を印づける)。これだけですでに、一種のつまみ食い(cherry-picking)は打ち破られている。ある規則をこっそり巻き戻して、後からその規則が最初から存在しなかったかのような履歴を提示する、ということとはできない。

構成性がそれ自体では届けないのが、出来事3——outcome-independence のもう半分——だ。却下された提案は状態を何も変えないので、chain に載る義務がそもそもない。そして、ある run がそもそも記録の対象になるかどうかは、今日では構築ではなく方針

の問題にとどまる。この種の出来事まで届くには、まだ構築によっては建てられていない記録のゲートが要る。§9 がそれを、開いた課題として印づける。

だから誠実な読みは狭い。しかし、それでも述べる価値がある。governance の境界では、記録は**何が効力を持ったか**について非裁量(non-discretionary)だ。出来事1と2は、curate(取捨選択)された証拠ではなく事実であり、それとともに governance の軌跡——どの規則が採択され、修正され、巻き戻されたか——もまた事実になる。「compute が行ったことを勝ち負けによらず何もかも記録する」という、より豊かな results 層の理想は、常設の保証ではなく、なお志向にとどまっている。

KairosChain は構成的な register を軸に設計されており、現在の開発サイクルにおける具体的な工学的内容は、まさに、記録保証を方針型から構築型へ、層ごとに**移行(migration)**することだ。ここでの議論にとって効いてくるのは **governance 層**——Skill が昇格され、instance 自身の constitution が改訂される層——であり、そこでは記録は by design で構成的だ。構築の理想にもっとも近い。変更は自分の chain 記載を通してでなければ effective と数えられないよう意図されており、だから effective な変更の class 全体が非裁量になる。

ここでさえ、理想は完成ではなく接近だ。現在の実装では、一部の変更はまずファイルに書かれ、その後で記録される。一つの分割不能な段ではなく二段になっているため、二段の間のクラッシュも、捕捉されてログに書かれるだけの記録失敗(変更は効力を持ったまま残る)も、effective な変更を chain の外にとどめうる。この両方の経路を閉じることが、移行のいま現在の仕事だ。

他の層は意図的に異なる体制を担う——選択的に attest される opt-in な改竄検出の層と、破棄されることが前提の使い捨て作業文脈の層——が、ここで巡回するのではなく、それぞれが属する場所で扱う。§6 がそれらを L2→L1→L0 の昇格経路に位置づけ、§9 がそれらの指し示す限界を引く。

一つの scope(範囲)の限定は、脚注ではなくここに属する。単一の運用者が自分のマシンで運用する chain において「irreversible(不可逆)」「append-only(追記専用)」と言うとき、それはその**運用者の誠実な運用に相対的な保証**だ。hash chain は、chain の最新 hash の信頼できる写しをすでに持っている者であれば誰に対しても、改竄を**evident(見える)**ようにする。だが、運用者自身が履歴を書き換え、編集箇所から先のすべての

hash を計算し直すことまでは、不可能には**しない**。運用者を誠実に保つのは **external anchoring(外部アンカリング)**——chain の状態を、運用者が制御しない記録に commit すること——であり、後出の §8 は、とりわけ、まさにそうした anchor の一つだ。

最後に、構成的記録はシステムの生きる**時間の種類**を変える。ログは **Chronos** に住む。時計の時間であり、入れ替え可能な瞬間が一つまた一つと連なり、そのどれも大きな意味の損失なく上書きできる。構成的記録は **Kairos** に住む。他のどの瞬間とも入れ替えのきかない、張りつめた決定的な瞬間を指すギリシャ語だ。先に述べた信頼の範囲内において、一つひとつの記載は、システムが何であるかの決定的で不可逆な再構成にあたる。もっとも近い哲学的な縁者は Whitehead (1929) の「objective immortality(客体的不滅性)」だ。完結した瞬間は次の瞬間に消去されず、後続するすべての瞬間が受け継ぐ恒久的な条件になる。

ありていに名指せば、これは実体(substance)と過程(process)の存在論の古い分かれ目にあたる。ログは、まず存在して、あとから痕跡を残すシステムを前提する。構成的記録は、システムを、その存在が記録された出来事の履歴であるような種類の実体にする。設計は過程の側を取る——governance 層で、そしてそこでだけ。execution 層では、自分が作らなかった基盤の上に載っていることを §4 がすでに認めている。

§3 の境界横断は、ひとたび記録されれば、システムが持つ情報ではない。システムがあるところのものの一部だ。

§4 と §5 は、一つの手として読むのが最もよい。§4 は、システムが統治するものと、ただその上で走るだけのものとの継ぎ目を**明示的**にした。§5 は、その継ぎ目を越える通行それぞれを記録することを、それが効力を持つまさにその行為にした——governance 層において **by construction**、§5 の migration がその保証をどこまで運んだかに応じた範囲で。通常のシステムがこの境界を暗黙のまま記録せずに置くのに対し、この作り方をした自己改訂システムは、境界を明示的にし、その通行を外から監査可能にする——システムが内側から自分の天井を見えるようになるのではなく(§2 がそれを禁じる)、外の観測者にとって読み取れるようになる、ということだ。通常のシステムでは沈黙している最外フレームの境界が、ここでは記録された境界になる。それが、§6 がこれから人間を組み込む、その境界である。

6. 境界としての人間の制度化

§2 と §3 は二つのことを確立した。第一に、agent は自分の最外フレームの外に立てない。第二に、人間もまた自分自身のフレームの外には立てないが、agent の盲点に働きかけることができ、しかも agent の状況をまるごと手中に握っている (§3 の力の非対称性)。そして §5 は、保持の形を確保した——effective な governance 変更は、記録の外にとどまれない。しかし記録の機構だけでは生み出せないものが残る。**中身**、すなわち「どの変更が効力を持つに値するか」という判断がそれにあたる。この判断をシステムが単独で下せば、それは自分のフレームの内側から下されたものにしかない。死角が死角を採点することになり、§2 の天井がそれを許さない。だから判断には、異なるフレームを持つ者——partial outside (§3)——の行為が組み込まれていなければならない。そして変更の触れる層が深いほど、その関与は重くなる必要があり、最も深い段——システム自身の constitution を改訂する段——では、関与を省くことができない。KairosChain はこの要請を、漠然とした善意から制度に変える。三つの機構による。

- **ループの「中」ではなく、境界の「上」に立つ人間。** 慣れ親しんだ語彙では、ここで求められているものを言い尽くせない。監督者 (supervisor) は見ていて、あとから口を出す——無視されることもあるし、「書き忘れ」と同じ形で破れもする。門番 (gatekeeper) でさえ、まだ証拠的な絵にとどまる。門番は門の外に立ち、通過する者を検分する係であって、居眠りもできる。ここで建てられているものは種類が違う。constitution を改訂する段では、人間の承認が、**変更が効力を持ちかつ記録される、その単一の段の部品**として組み込まれている (§5)。承認が無いとき、門が閉じているのではない——門が存在しない。「承認なしに効力を持った」という許容される状態そのものが無い。これは constitutive recording を人間の行為にまで延長したものにほかならない。銀行の貸金庫がこの像を運ぶ。扉は、銀行員の鍵と利用者の鍵が同時に回るときだけ開く。この銀行員を、監視カメラの前に座る警備員と比べてみてほしい。警備員は見ていて、あとから報告できるが、居眠りもできるし、彼がいてもいなくても扉は開く。二本鍵の銀行員は、出来事を見守っているのではなく、**出来事の部品**として働いている。constitution の段における人間の承認が、その第二の鍵にあたる。(より浅い層では、変更は人間の承認なしに記録され効力を持ちうる。関門が構成的に組み込まれているのは constitution の段だけであり——それがどの段かは次の機構が示

す。)したがって人間は、外から見守る監督者としてモデル化されるのでもなければ、内側に取り付けられたもう一つの部品としてモデル化されるのでもない。人間は境界の上に (*on the boundary*) 立つ。人間の行為——変更を承認する、異議を唱える、組み替える——が、システムの自己再定義がそこを通過する出来事そのものとなる。システムの境界は、部分的にそれらの行為でできている。流行りの「human-in-the-loop」という言い方は、このことを言い足りない。問題のループとはシステムが自分を定義するループであり、人間はその荷重を担う部分の一つとして機能している。

- **外部を内部化する昇格経路。** 外部の洞察——レビュアーの指摘、あるいはモデルが自分のフレームの内側からは決して到達できなかった組み替え——に何が起きるかを考えてほしい。KairosChain では、それはまず使い捨ての作業文脈 (L2 層) として入る。価値があると分かれば、精錬され再利用可能な知識 (L1) に蒸留される。そしてやがて、システム自身の constitution (L0) に書き込まれうる。各昇格は chain に記録され、constitution そのものへの書き込み (L1 → L0 の昇格) では人間が関門となる (human-gated)。具体例を挙げる。レビュアーが設計の盲点を指摘する。その観察は、まず一つのセッションの中のメモにすぎない。次にシステムが参照する保存された規約になり、最後に——それに値すれば——システムが従う規則になる。この経路は、人間によって外部化された *agent* の盲点が、次の機会には *agent* 自身のフレームの一部になるための常設の機構として機能している。フレームの天井が単一の英雄的行為として禁じるもの——自己超越——は、この経路によって近づかれるのではない。超越に「手前」はなく、「半分だけ外側にいる」という状態もない。そうではなく、自己超越は**機能的に代替される (functionally substituted)**。反復され、記録される二者の過程が、超越的な立ち位置が与えたはずのものの一部を、誰もそこに立つことなく与える。

- **内省では供給できないものを供給する harness。** §2 の議論により、LLM は、自分が今どの層で行為しているかを内省だけでは確実に判定できない。これは訓練で消せる欠陥ではなく、天井がもたらす直接の帰結にあたる。それを内側から知ろうとすれば、§2 が否定したまさにその外側の視点を必要とするからだ。そこで harness がその情報を外から供給する。いまどの層が作動しているか、ある行為がどの境界を横断するかを、明示的な文脈として注入する。分業は原理的なものとなる。モデルはフレーム

内の能力をもたらし、harness はフレームの縁がどこにあるかについての知識をもたらし、人間は partial outside をもたらし。

この制度が生み出すものについて、二点、正確に述べておく価値がある。第一に、ここで生まれるのは、人間のフレームが広がったものでも、モデルのフレームが広がったものでもない——どちらも自分のフレームの外には出られない (§3)。生まれるのは**第三のもの**である。人間とモデルと記録とで構成される合成システムであり、それ自身の outermost frame を備えている。複数のフレームが融合してより高い天井になったわけではない。内側がひとつ増えたのであり、その内側にもやはり天井がある——§9 が末尾に据え、欠陥としてではなくエンジンとして読み直す regress (後退) がそれである。第二に、境界の仕事は双方向に走る。越境が人間の死角に着地し、受け取られるとき、変わるのは人間が持つ情報の在庫ではなく、ものの見え方そのものの形である——§5 が述べた「なる」の人間側にほかならず、身体における「なる」がそうであるように、暗黙のうちに起こる。chain はこの共同の履歴のうちシステム側の半分を、第三者が検分できる形で保持する。人間側の半分は、chain にはまったく保持できない——§7 がその非対称性を帰結 (corollary) へと変える。

これらのどれひとつとして、§2 の天井を解消してはいない——天井自身の論理により、何ものにも解消できないからだ。これらが行うのは、天井を、沈黙したまま壊れる故障様式から、横断の往来が記録される明示的で計装された (instrumented) 境界へと変えることである。構えは征服ではなく受容であり——工学によって形を与えられた受容は、結果として、あきらめではなく生産的なものとなる。

7. 経済的な帰結 (corollary): 資本としての経験

§5 は一つの存在論的な論点で幕を閉じた。構成的記録 (constitutive recording) の下では、システムがあるところのものとは、記録された出来事の履歴そのものにほかならない。本節ではその論点の経済的な側面を描く。システムのあるが記録された軌跡に等しいなら、その価値もまた軌跡に宿る——しかもその価値は、複製できる仕組み (mechanism) の側にではなく、軌跡そのものの側にある。これこそ、この設計が積み上げた**経験を資本 (capital) に変える**という表現が指し示す内容にほかならない。資産とは、

機械でもなく、記録それ単独でもなく、境界横断 (§3) を anchor で留めた軌跡である。最も近い経済上の縁者は **goodwill(暖簾)**——継続企業が有形の構成要素を超えて帯びる無形の割増——だが、一点で決定的に異なる。暖簾は主張され、見積もられるものであるのに対し、構成的な軌跡は監査される。

この主張は注意深く述べる必要がある。hash で連鎖された記録は、あらゆるファイルと同じようにビット単位で複製できるからだ。構築 (by construction) が保証するのは、軌跡が複製不能であることではない。複製が *continuation* (継続) にはなれない、という点だ。仕組みと chain をまるごと複製した者が手にするのは、あなたの複製ではなく **fork (分岐)** である。複製の瞬間に限って言えば、§5 の同一性テーゼにより、複製はオリジナルと同じ constitution を持つ——ビットは同一であり、両者を区別するものはまだ何もない。したがって originality (本物であること) は、バイト列の性質ではなく、その後続く *anchor* 付き軌跡の性質として立ち現れる。両者がそれぞれ自分の境界横断を記録し始めた瞬間から、二つの constitution は分岐し、「どちらが本物か」という問いは**主張ではなく監査 (audit) の対象**となる。決着をつけるのは chain 単独ではない——chain はどちらの複製についても同じように見えるからだ。決着をつけるのは、§5 の external anchor——§8 で具体化される DOI とタイムスタンプであり、各々がその瞬間までの chain の prefix を公的な身元・公的な時刻に留め置く——と、運用者自身の継続する仕事とである。chain は各軌跡を帰属可能かつ改竄検出可能にし、anchor はそのうちの一方を公的な身元に結びつける。両者が合わさって初めて *provenance* (来歴) が成立する。

§5 の governance 境界における非裁量性は、その provenance に独特の強さを与える——ただし、実際に稼いだ分だけだ。懸念の最も鋭い形にはきちんと向き合う価値がある。それは「誰かが複製するのでは？」ではなく「誰かが同じくらい良く見える対抗品を捏造するのでは？」という問いだ。まず誠実な場合を考える。自分の chain を誠実に走らせてきたオリジナルは、その途上で、実際に効力を持った修正や巻き戻しをこっそり落とすことができなかった——governance 層の内側では、それらは by design で非裁量だからだ。したがって、記録された *governance* 軌跡は *retroactively* (事後的に) *curate* された編集ルールではない。採択して後から取り消した規則があれば、採択も取り消しも両方が chain に載っており、事後に消し去ることはできない。(提案段階での *curate* は依然として可能だ——運用者が、きまり悪い変更を一度も効力に至らせなければよい——)ので、保

証の中身は、*effective* な記録が消去不能だということであって、きまり悪いものが一度も試みられなかったということではない。)これは「これまで犯したあらゆる失策」の記録より狭い資産である——行き止まりに終わった results 層の記録こそ、§9 が開いたままにする、まさに選択的で使い捨ての素材だ——が、それでも実在する資産であり、対抗者が単に言い張って消し去れるようなものではない。

不誠実な場合にこそ、この仕組みの力が実際に発揮される。そしてその力は、記録が outcome-independent であることから来るのではない。§5 が認めるとおり、単一の運用者であればローカルの chain を書き換えて hash を計算し直すことができる以上、記録の内部構造だけでは失敗の偽造を防げるものは何もない。力の源泉は **external anchoring** にある——ただし、主張が行きすぎも足りなさすぎもしないよう、二つの限定を加える。第一に、anchor は点的(punctate)である。§8 の deposit が留め置くのはその瞬間までの prefix であり、したがって経済的に価値ある区間——deposit の後に蓄積された経験——は、運用者がその後に行う deposit が anchor する分だけしか anchor されない。持続的な provenance とは、一回の行為ではなく、anchor の cadence(律動的な反復)なのである。第二に、ここが構成的設計と素のタイムスタンプとの差が効いてくる箇所である。arXiv への投稿、署名付き git tag、公証人の印——これらはすべてスナップショットを anchor する。すなわちこれこれのバイト列がこの時刻に存在したことを証明するだけであり、それ以上のことは言えない。commit した内容の傍らで、別の規則を保持したり別のコードを走らせたりしていた可能性がツネに残るからである。構成的な governance chain は、ひとたび anchor されれば、より強い証明力を持つ。設計上、chain を通さない *effective* な governance 変更は存在しないためである。anchor された prefix が証するのは、単にある時刻のあるバイト列ではなく、記録された governance の class については、その時刻までの *effective-governance identity* の完全性(*the complete effective-governance identity up to that time*)である——対抗者にとっては、後から明かせる「記録され効力を持った変更」が残らず、オリジナルにとっては否認すべきものが残らない(§5 の定義による記録された governance の class については完全であり、*behavioural* な class については、write-then-record の隙間を migration がどこまで塞いだかに応じた分だけ完全である)。この完全性こそが、汎用のタイムスタンプでは再現できないものにほかならない。このプレミアムは、厳密に言えば hash chain に固有のものではない——commit した内容だけを deploy するという規律があれば、同じ record↔effect の閉じ方に近づくことはできる。構成的設計

が寄与するのは、その結合を、守ることを覚えていなければならない手続きではなく、**構造的 (structural)** なものにする点にある。したがって守れる資産とは、秘匿でもなければ、単独の chain でもない。それは、完全で、事後的に curate し直すことのできない governance 軌跡であり、競争相手が事後には偽造しえない公的な anchor に留め置かれたものである。

これはまさに、fork された code repository の状況と同じである——プロジェクトの変更履歴をまるごと完全に複製し、そこから開発を独立に続けられるようにしたものだ。fork は履歴を完璧に複製する。複製できないのは、どの fork をコミュニティが、メンテナが、署名されたリリースが、系統 (line of descent) の本流として認め続けるか、という点である。ビットは共有されるが、系統は帰属によって決まる。(仕組みだけを chain なしで複製するとなると話はさらに露骨だ。手に入るのは、constitution が空のシステム——借り着をまとった、履歴ゼロの新生児——だけである。)

8. 自己埋め込み: 自らの実演としての本文書

記録は**構成的**だと論じるノートが、自分自身については**証拠的**にとどまるなら、主張と実践が乖離する。だから本文書は、自らが記述するまさにそのシステムへ書き込まれる——貢献についての外部の注記としてではなく、**instance 自身の knowledge に L1 参照用 Skill として登録され、その chain に記録される**ものとして。そうすることで、本文書はその instance が何であるかの一部になる。手順はここで定め、deposit する版で実行する。

deposit される成果物は、凍結した**1つのファイル**である: 英語(正本)の Markdown ソースから、この作業用の editorial-status 注記を除いたもの(あれは足場であって deposit の一部ではない)で、改行を LF に、文字を Unicode NFC に正規化しただけのものだ。束縛されるのは、その**ファイルの生バイトの SHA-256** である——正準形をこしらえる操作も、空にすべき欄もない。描画した PDF と、併載する日本語訳は便宜写しであり、食い違いがあれば英語ファイルのバイトが優先する。

文書に埋め込むのは、hash より**前に**確定する handle だけである(埋め込んでも hash が動かないように)。三つあり、いずれも内容非依存だ:

- 予約した Zenodo の **version DOI**、
- リポジトリ内の**本文書ソースファイルへの直接リンク**(tag もしくは branch のパス——内容依存になる commit-SHA の permalink は用いない)、
- **検証アドレス(verification address)**——このファイルを係留する記録が登録される、安定な handle。

手順は次のとおり。

1. **凍結して hash する。** 三つの handle を埋め込んだ凍結 deposit ファイルを作り、その生バイトの SHA-256 を取る。
2. **構成的にコミットする。** その hash を元の KairosChain instance の chain に記録する——L1 参照用 Skill として保持される本文書を instance のあるところへ入れる構成的記録である。記録は、**検証アドレス**をファイルの hash に、DOI とリポジトリ参照とともに結びつける。文書が record identifier を受け取って書き戻すことはない——文書は何も抱えない。
3. **公開し、実演する。** 凍結ファイルを、予約した DOI のもとで Zenodo に deposit する——これが**公開参照先**であり、運用者が制御しない記録である以上、§5 の **external anchor(外部係留)**でもあり、コミット済み record までの chain の prefix を留め置く。共有 Meeting Place には、対応する **attestation**——検証アドレスに束縛されたダイジェストで、Zenodo とリポジトリの写しを指す——を登録する。本体は Zenodo にあり、Meeting Place はこの参照記録だけを持つ。これが**構成的記録の実演場所**であり、本文書を federation に入れる。この運用者自身の chain にとって、その attestation は同一運用者の支配下にある公開参照であって、さらなる external anchor ではない。だが別の運用者の instance にとっては、同じ attestation がそれ自体で真の external anchor になる——係留の役割は関係的である(§5)。

なぜ自分の尾を追わずに済むのか。懸念はもっともらしく見える——文書は記録され、記録は文書を指す——が、その参照は**一方向**、記録から文書へだけ走り、文書から自分自身の指紋へは決して走らない。文書は自分の hash も record identifier も抱えず、hash より前に確定した三つの内容非依存 handle だけを抱えるので、そのバイトの中に解くべき自己言及は何もない。第三者は Zenodo からファイルを download し、その SHA-

256 を取り、検証アドレスがたどり着く attestation が持つ hash と突き合わせる——指紋を独立に計算し直すこの手続きが、確認を循環でなく意味あるものにする。残るループは意図されたもの、まさに Hofstadter の意味での strange loop である: constitutive recording についての文書が constitutive に記録され、その公開という行為 (§7 が安全だと論じたまさにその行為) が、instance を構成する記録された出来事の一つになる。ループが宿るのは記録するという行為であって、文書が印字する番号ではない——だからこそ文書は自分自身について何も抱えなくてよい。

弁解ではなく微笑とともに記しておく——この節もまた §2 を逃れない。文書を chain に埋め込むことは、システムのフレームの内側のもう一つの手であって、外への一歩ではない。それこそが要点である。

§5 自身についての区別もまた、例外ではない。§5 がその区別を引いた以上、ここで明言しておく価値がある。この埋め込みが実演しているのは *constitutivity* (構成性)——chain に結びつけられ、公的に anchor された、単一の完結した成果物としての性質——であって、*outcome-independence* ではない。このノート自身の成り立ちは、むしろその対極にある。v0.1 から本版に至る drafting の履歴、多周にわたるレビューが残したあらゆる巻き戻し——それらはまさに、§5 が使い捨ての層へ割り当てた、選択的に curate され anchor されない素材そのものである。deposit されるのは生き残った draft であり、生き残らなかった draft の記録ではない。したがって、この区別を論じている文書自身が、その構築過程においては区別の弱い側の一例となっている——選択的に記憶された過程から引き出された、構成的に commit された結論である。それはこの文書が立つべき正直な場所であって、隠すべき矛盾ではない。システムについてのノートが、システム自身の生きている非対称性を、もう一度演じているにすぎない。

9. 限界と開かれた問い

- **範囲。** 本ノートは設計ノートであり、新たな実証的主張を含まない。最外の outermost frame がもつ天井 (§2) は strange loop の系譜にもとづく論証であって、測定ではない。その限界についての実証研究は、準備中の姉妹ノートで扱う予定であり、また我々としては、2009 年に公開済みの装置を用いた独立の再現がなされ

ることを望んでいる。§§4-6 の設計上の答えは、それとは種類が違う——紙の上で証明されるものではなく、運用のなかで実証されることを意図している。各 effective な governance 変更がその記録を通してのみ効力を持つ、その KairosChain 自身の継続的な稼働こそが、本ノートのテーゼが単に主張されるのではなく裏づけられる場所であることを、我々は願っている。

- **境界の役割も例外ではない。** §6 は人間を境界に据えるが、人間の行為もまたフレームに縛られている (§3 の対称性)。この設計は、人間が「view from nowhere」を供給すると主張しない——人間が供給するのは別のどこか (*elsewhere*) からの眺めにすぎず、それを支えるのは §3 が認めた埋め込みをめぐる力の非対称性である。境界の役割の一部を、十分に異なるフレームをもつ別の agent——別システムのモデルや、分岐した記録履歴をもつ別の instance——が担えるかどうかは、いまだ開かれた問いであり、システムの federation 層はまさにそれを試すために設けられている。
- **選択的 attestation は outcome-independence ではない——**
governance 層もまた、まだそうではない。 構成的な governance 層と使い捨ての作業文脈層のあいだに、最近つくられた**中間の instrument (道具)**が位置する。個々の作業文脈単位に対する *opt-in* 方式の改竄検出層である。人間が、検分に耐える価値のある作業メモを選び出すと、その内容の digest がただちに追記専用・hash 連結の proof store (証明の保管庫) に固定される。後に訂正が加われば、新しい証明書が古いものの上に重ねて発行される——**非破壊的な supersession (上書き継承)**であり、消去は決して起こらない。この仕組みは、検分可能性を裁量の代償で買う。何を attest するかを人間が決める以上、attestation がないことは何も証明しない。まさに §5 が警告した弱点である——しかも本稿執筆時点では、この層はまだ外部に anchor されておらず、その保証は §5 の「誠実な運用」モデルの内側でしか成り立たない。使い捨て文脈の層にとって、それは正しい trade (取引) である。保存された文脈に淘汰をかけることこそが、この層の眼目だからだ。しかし、より深い限界がある。完全な outcome-independence——失敗した試みも、結果がどうであれ実行された run も記録すること——は、今日、どこにおいても by construction では届けていない。governance の境界でさえ事情は同じである。governance の境界は、より狭い目標——effective な変更を隠せないこと——だけを守るよう設計されており、しかもそれすら §5 の移行が閉じた範囲でしか守れない。線をどこに引くべきか

——どの class の行為が結果によらず記録されねばならず、どの行為が人間の選択に委ねられてよいか——は、決着した問いではなく、生きた設計上の問いである。そしてそれは、あらゆる実行時の記録ゲートがいずれ裁定しなければならない境界問題でもある。意味ある否定的結果は outcome-independent な体制に値し、純粋なノイズ（構文エラー、中断されたキー入力）は使い捨ての体制に属する。両者のあいだの線は、まだ by construction では引かれていない。

- **後退(regress)は織り込み済みである。** 外部を取り込むどの制度も、新たな内部をつくり出し、その内部の外部はまた外から供給されねばならない。我々はこれを欠陥ではなく原動力として読む。完全な自己記述が原理的に得られない以上、システムの進化は、自分についてまだ記述できないものの恒久的な残余によって駆動される。昇格経路は後退を終わらせない——後退を動力に変える。
- **一般性。** 議論が一つのシステム(KairosChain)に即して述べられているのは、constitutive recording が、システムが by construction でもつか・もたないかのどちらかである性質であり、抽象的に主張できるものではないからである。設計の型——同一構造の自己言及、構築水準の記録、制度化された境界——は再利用のために提示する。その再利用が元の設計を薄めない理由は、§7 の帰結として示されている。

謝辞

本稿の草稿に有益なコメントを寄せてくださった、金沢工業大学の Takeshi Konno 氏、沖縄工業高等専門学校の Takashi Sato 氏に感謝する。誤りはすべて著者の責に帰する。

References

(書誌情報のため原文のまま。訳出しない。)

- Bürgerliches Gesetzbuch (BGB) §873 (German Civil Code): acquisition of rights in land takes effect upon entry in the land register.
- Dennett, D. C. (1984). Cognitive Wheels: The Frame Problem of AI. In C. Hookway (ed.), *Minds, Machines and Evolution*. Cambridge University Press.
- Dreyfus, H. L. (1991). *Being-in-the-World: A Commentary on Heidegger's Being and Time, Division I*. MIT Press.
- Fowler, M. (2005). Event Sourcing. martinowler.com.
<https://martinfowler.com/eaDev/EventSourcing.html>
- Hatakeyama, M., & Hashimoto, T. (2009). Minimum Nomic: a tool for studying rule dynamics. *Artificial Life and Robotics* 13(2): 500–503.
<https://doi.org/10.1007/s10015-008-0605-6>
- Hofstadter, D. R. (1979). *Gödel, Escher, Bach: An Eternal Golden Braid*. Basic Books.
- Horibe, K., Hatakeyama, M., Masumoto, G., Hashimoto, T., & Romero, P. (2026). Scale-Dependent Collective Adaptation in Self-Amending LLM Societies: A Cross-Family Study of Emergent Governance.
arXiv:2605.17510.
- Maturana, H. R., & Varela, F. J. (1980). *Autopoiesis and Cognition: The Realization of the Living*. Reidel.
- McCarthy, J., & Hayes, P. J. (1969). Some Philosophical Problems from the Standpoint of Artificial Intelligence. *Machine Intelligence* 4: 463–502.
- Nagel, T. (1986). *The View from Nowhere*. Oxford University Press.
- Polanyi, M. (1966). *The Tacit Dimension*. Doubleday.
- Searle, J. R. (1995). *The Construction of Social Reality*. Free Press.
- Suber, P. (1990). *The Paradox of Self-Amendment*. Peter Lang.
(Nomic, Appendix.)

- Varela, F. J., Thompson, E., & Rosch, E. (1991). *The Embodied Mind: Cognitive Science and Human Experience*. MIT Press.
- Whitehead, A. N. (1929). *Process and Reality*. Macmillan.